

Parameter-Space Trajectory Metrics

Ben Meyers

Overview

Gradient descent is a physical process. Parameters move through a high-dimensional space under forces determined by the loss landscape, the architecture, and the update rule. Standard training diagnostics — loss curves, accuracy, gradient norms — are scalar projections of this movement that discard almost everything.

The framework below instruments the trajectory itself: how far did each parameter travel, in what direction, driven by which gradient sources, and how much of that travel was purposeful versus wasted? Three leverage metrics then weight those trajectories by different notions of parameter importance.

The guiding principle throughout is that *gross and net are not redundant*. Their gap is where the diagnostic value lives. Whenever we extend a quantity to multiple sources or to leveraged variants, we preserve this gap on purpose; partition is not a goal of the framework, and the moments where naive partition breaks (when sources disagree in sign, when sources cancel exactly) are exactly the moments the framework is built to surface.

1 Base Accumulation: Gross Path and Net Displacement

Let $\theta_i(t)$ denote the value of parameter element i at training step t , and let $\Delta\theta_i(t) = \theta_i(t) - \theta_i(t-1)$ be the update applied at step t .

Gross path length.

$$G_i = \sum_t |\Delta\theta_i(t)| \tag{1}$$

This is the odometer: it accumulates every step’s magnitude regardless of direction. A parameter that walks $+0.1$, then -0.1 , then $+0.1$, then -0.1 has $G_i = 0.4$.

Net displacement.

$$D_i = \sum_t \Delta\theta_i(t) = \theta_i(T) - \theta_i(0) \tag{2}$$

This is where the parameter ended up minus where it started. The wandering parameter above has $D_i = 0$.

The G – D gap as the foundational diagnostic. The framework rests on a single observation: $G_i \geq |D_i|$, with equality only when the parameter walks a monotone straight line. *The gap between them is the quantity of interest.* Every higher-level metric in this document inherits this design — gross quantities measure work done (including wasted, oscillatory, or fought-over work), net quantities measure work committed to the trajectory, and their ratios surface different kinds of inefficiency.

Network-level efficiencies: two distinct ratios.

$$\eta_1 = \frac{|D|_1}{G} = \frac{\sum_i |D_i|}{\sum_i G_i} \in [0, 1] \quad (3)$$

$$\eta_2 = \frac{|D|_2}{G} \in [0, 1] \quad (4)$$

Both are kept because they answer different questions, and the gap between them is itself informative.

The L1 ratio admits a clean decomposition:

$$\eta_1 = \frac{1}{G} \sum_i |D_i| = \sum_i \frac{G_i}{G} \cdot \eta_i \quad (5)$$

So η_1 is the path-length-weighted mean of per-parameter efficiencies: *on average, across the moving parts, how purposeful was the motion?*

The L2 ratio treats the parameter vector as one object in high-dimensional space. $|D|_2$ is the Euclidean straight-line distance from $\theta(0)$ to $\theta(T)$. So η_2 asks: *did the network as a whole travel a geodesic, or did it curve through space?*

The two decouple distinct failure modes:

- η_1 low, η_2 higher: individual parameters oscillate, but the oscillations cancel coherently across coordinates — different parameters zig-zag in coordinated ways such that the joint vector still lands somewhere meaningful.
- η_1 higher, η_2 low: each parameter moves fairly directly, but the collective trajectory curves — early training pulled the vector one way, later training another, and net displacement is small relative to total path despite each coordinate being individually well-behaved.

Per-parameter efficiency.

$$\eta_i = \frac{|D_i|}{G_i} \in [0, 1] \quad (6)$$

Per-parameter directedness. $\eta_i \rightarrow 1$: the parameter walked a near-straight line. $\eta_i \rightarrow 0$: it churned without going anywhere. Low efficiency arises from conflicting gradients, minibatch noise, or genuinely curved geometry — all useful to know about, even if not all problematic.

1.1 Per-Source Attribution

In a typical training setup, the gradient at each parameter is itself a sum of contributions from different sources — different loss terms, different attention heads, different data subsets, auxiliary objectives, regularizers — anything for which a separate gradient can be computed.

$$g_i(t) = \sum_{s=0}^{S-1} g_i^{(s)}(t) \quad (7)$$

We split each step’s update among sources in proportion to their share of the combined gradient:

$$\Delta\theta_i^{(s)}(t) = \frac{g_i^{(s)}(t)}{g_i(t)} \cdot \Delta\theta_i(t) \quad (8)$$

Why this is the right attribution. The signs handle themselves elegantly. If source A pushes $+2$ on a parameter whose net combined gradient is -5 (because other sources collectively pushed -7), A 's share is $-2/5$. This is correct: A wanted the parameter to go up, but it went down, so A 's contribution to the actual displacement is negative. Eq. (8) captures both magnitude and sign of disagreement in one ratio.

The shares sum to 1, so the per-source updates partition the actual update exactly: $\sum_s \Delta\theta_i^{(s)}(t) = \Delta\theta_i(t)$ as signed numbers.

Validity under joint Adam. This framework assumes all source gradients are summed *before* the optimizer sees them, so a single Adam state (m_i, v_i) tracks the joint signal and emits one $\Delta\theta_i(t)$ per parameter per step. Under this setup, eq. (8) is *exact at every step*: Adam applies one scalar to the combined gradient, and that scalar distributes over the pre-Adam sum. The shared Adam state is not a confound to be undone — it is the medium the sources are jointly carving and being shaped by, which is the object of study.

Numerically stable computation. Computed naively, eq. (8) becomes ill-conditioned when sources nearly cancel: $g_i^{(s)}/g_i$ is a ratio of small numbers in float32. Reorganize the algebra:

$$\Delta\theta_i^{(s)}(t) = g_i^{(s)}(t) \cdot k_i(t), \quad k_i(t) = \frac{\Delta\theta_i(t)}{g_i(t)} \quad (9)$$

Mathematically identical to eq. (8), but numerically much better behaved. The factor $k_i(t)$ is Adam's effective per-step gain at this parameter — a ratio of correlated quantities that stays well-conditioned across regimes. The poorly-conditioned ratio $g_i^{(s)}/g_i$ never appears.

When $k_i(t)$ is unusually large (small g_i , normal-sized $\Delta\theta_i$), it indicates Adam is in a momentum-carry regime at this parameter — today's motion is being driven by accumulated history rather than by today's gradient. Per-source attributions in this regime can be larger in magnitude than naive intuition suggests, because tiny current pushes are scaled by Adam's high local gain. This is correct, and worth surfacing as its own diagnostic rather than suppressing.

Edge case: $g_i(t) = 0$. When the joint gradient is exactly zero, $k_i(t)$ is undefined. Two cases collapse here:

- All sources are zero: nothing pushed, nothing to attribute. Skip the step's update to the per-source accumulators.
- Sources are nonzero and cancel exactly: today's joint update (if any) is being driven entirely by Adam's history — by gradients from prior steps that were already attributed to their sources on the steps when they originally occurred. Attributing today's update again would be double-counting. Skip the step's update to the per-source accumulators.

In both cases, set $k_i(t) := 0$ and continue. No information is lost: real source activity in the cancellation case is recovered by the sidecar accumulator below.

Per-source accumulators.

$$D_i^{(s)} = \sum_t \Delta\theta_i^{(s)}(t) G_i^{(s)} = \sum_t \left| \Delta\theta_i^{(s)}(t) \right| \quad (10)$$

$D_i^{(s)}$ is signed — it accumulates the signed shares from eq. (8), including negative shares that occur when source s pushes against the joint direction. By construction $\sum_s D_i^{(s)} = D_i$ exactly.

$G_i^{(s)}$ accumulates absolute shares. Crucially, $\sum_s G_i^{(s)} \geq G_i$ in general, with equality only when all active sources agree in sign at every step. *This inflation is a feature, not a bug.* It is the source-level analog of the foundational $G \geq |D|$ gap: at the network level, $G \geq |D|$ measures committed waste (oscillation that survived into the trajectory); at the source level, $\sum_s G_i^{(s)} \geq G_i$ measures source discord (cancellation between sources fighting at the same parameter at the same step).

Sidecar: pushed gradient. The eq. (8) accumulators are silent on steps where $g_i = 0$ (all-cancellation). To capture source activity at those moments, accumulate raw gradient magnitudes independently:

$$P_i^{(s)} = \sum_t |g_i^{(s)}(t)| \quad (11)$$

This is unrelated to the optimizer; it is a pure record of how hard each source pushed at each parameter, in raw gradient units. It runs every step, including pure-cancellation steps that the eq. (8) machinery skips. $P_i^{(s)}$ is not a partition of anything — $\sum_s P_i^{(s)} \neq G_i$ in any natural unit — and it is in different units from $G_i^{(s)}$ (gradient-space rather than trajectory-space). Its role is comparative: across sources at one parameter, across time at one source, or against $|D_i^{(s)}|$ to ask “how much of all that pushing translated into committed motion?”

1.2 The Three Disentangled Ratios of Waste

The framework yields three distinct ratios, all variants of “gross over net,” each measuring a different scale of waste:

Ratio	What it measures
$G_i/ D_i $	Committed waste — oscillation that survived into the trajectory
$G_i^{(s)}/ D_i^{(s)} $	Per-source directional incoherence — one source reversing itself
$\sum_s G_i^{(s)}/G_i$	Discord — sources fighting each other relative to actual motion

These are independently meaningful. A parameter can have low committed waste ($G_i \approx |D_i|$) but high discord ($\sum_s G_i^{(s)} \gg G_i$): sources are arguing intensely but their net always points the same way, so the parameter walks a straight line through controversy. Conversely, a parameter can have low discord (one source dominating) but high committed waste (that source itself oscillating). Disentangling these is the point.

Per-source per-parameter efficiency.

$$\eta_i^{(s)} = \frac{|D_i^{(s)}|}{G_i^{(s)}} \in [0, 1] \quad (12)$$

Per-source directedness. A source with high $G_i^{(s)}$ but low $\eta_i^{(s)}$ pushed hard at this parameter but kept reversing itself — either fighting other sources whose effects bled into its attribution, or genuinely oscillating in its own gradient signal. A source with high $\eta_i^{(s)}$ is consistently directional. The cumulative sign of $D_i^{(s)}$ tells you which way that source has been trying to move the parameter on net.

2 Metric I: Positional Leverage

$$\lambda_i^{\text{pos}} = \frac{|j : j \text{ is downstream of } i|}{P} \in [0, 1] \quad (13)$$

Pure graph topology: the fraction of parameters that depend on θ_i through the computation graph. Value-independent (does not look at the parameter’s value) and loss-independent (does not look at gradients). A weight in an early layer feeds into many downstream computations and has high λ^{pos} ; a weight in the final layer has low λ^{pos} .

Leveraged path length.

$$\tilde{G}_i^{\text{I}} = \lambda_i^{\text{pos}} \cdot G_i \quad (14)$$

This is useful as a baseline — it tells you which parameters are doing motion in structurally significant locations — but it is coarse. It cannot distinguish a critical early-layer weight from a redundant one in the same layer.

3 Metric II: Instantaneous Functional Influence

$$\lambda_i^{\text{fn}}(t) = \left| \frac{\partial f}{\partial \theta_i} \right|_2 \quad (15)$$

The norm of the Jacobian of the network’s output with respect to this parameter, at the current parameter values. *If I nudged θ_i a little right now, how much would the network’s output change?*

Value-dependent and live: this captures the realized behavioral influence of the parameter at this point in training. A weight in a saturated regime, or in a dead neuron, has low λ^{fn} regardless of where it sits in the graph.

The chain rule connects this directly to the loss gradient:

$$\frac{\partial L}{\partial \theta_i} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial \theta_i}, \quad \lambda_i^{\text{fn}} = \sqrt{\sum_{j=1}^p \left(\frac{\partial f_j}{\partial \theta_i} \right)^2} \quad (16)$$

So functional influence is the “structural half” of the loss gradient — the part depending on the parameter’s role in the function, separated from the part depending on the loss.

$$\tilde{G}_i^{\text{II}}(t) = \lambda_i^{\text{fn}}(t) \cdot G_i(t) \quad (17)$$

4 Metric III: Structural Potential

$$\lambda_i^{\text{str}} = \mathbb{E}_{\theta \sim P_{\text{init}}} \left[\left| \frac{\partial f}{\partial \theta_i} \right|_2 \right] \quad (18)$$

Same Jacobian norm, but averaged over random initializations. This strips out the current parameter values: *how influential is this parameter’s position in the architecture, on average, before training has done anything?*

Estimated by Monte Carlo over K random reinitializations:

$$\lambda_i^{\text{str}} \approx \frac{1}{K} \sum_{k=1}^K \left| \frac{\partial f^{(k)}}{\partial \theta_i} \right|_2 \quad (19)$$

$$\tilde{G}_i^{\text{III}} = \lambda_i^{\text{str}} \cdot G_i \quad (20)$$

4.1 Realized-to-Potential Ratio

$$\rho_i(t) = \frac{\lambda_i^{\text{fn}}(t)}{\lambda_i^{\text{str}}} \quad (21)$$

Is this parameter currently living up to its architectural potential? This is probably the richest derived quantity in the framework:

- $\rho_i \approx 1$: the parameter is being used roughly as the architecture intends.
- $\rho_i \ll 1$: the parameter has architectural potential but is not currently exercising it. Could be a dead unit, a saturated activation, or a feature waiting to specialize.
- $\rho_i \gg 1$: the parameter is exercising more functional influence than its architectural average — it is in a “hot” configuration.

This rhymes with neural tangent kernel intuitions about lazy versus feature-learning regimes. Parameters with ρ_i stuck near initialization values throughout training are probably in the lazy regime; parameters with ρ_i that swings significantly are doing feature learning.

5 Leverage-Weighted Source Attribution

Combining per-source motion and per-parameter leverage gives leverage-weighted versions of the attribution questions. Let λ denote any of the three leverage choices (positional, functional, or structural; the framework supports all three).

$$B^{(s)} = \sum_i \lambda_i \cdot G_i^{(s)} \quad (22)$$

The total leveraged motion source s has produced. A source that pushed hard at low-leverage parameters has small $B^{(s)}$; a source that pushed at high-leverage parameters has large $B^{(s)}$.

$$\tilde{D}^{(s)} = \left| \lambda \odot D^{(s)} \right|_2 \quad (23)$$

Leveraged net displacement attributable to source s , in the L2 sense (parallel to η_2).

$$\tilde{\eta}^{(s)} = \frac{\tilde{D}^{(s)}}{B^{(s)}} \quad (24)$$

Per-source leverage-weighted directedness.

Two normalizations of the structured-source share. A natural quantity to ask about is “how much of the leveraged work is being done by a designated subset of sources (e.g., structured heads)?” Two distinct normalizations are useful, and they answer different questions:

$$\phi = \frac{B^{(\text{structured})}}{\sum_s B^{(s)}} \quad (25)$$

Structured heads’ share of total demanded leveraged motion. The denominator $\sum_s B^{(s)}$ inherits the discord inflation from $\sum_s G_i^{(s)} \geq G_i$: when sources fight, $\sum_s B^{(s)} > \sum_i \lambda_i G_i$. So ϕ is implicitly normalized against *all the work all sources pushed for, including cancelled work*. This is the right

denominator if you want to ask “of the total push happening in the network, what fraction is coming from structured heads?”

$$\psi = \frac{B^{(\text{structured})}}{\sum_i \lambda_i \cdot G_i} \quad (26)$$

Structured heads’ demanded leveraged motion as a fraction of actually committed leveraged motion. The denominator is the joint leveraged path length — the trajectory’s actual leveraged motion, with no source-level inflation. ψ can exceed 1, and when it does, this is informative: structured heads pushed for more leveraged motion than the trajectory committed to, meaning their pushes were partially cancelled by other sources. The gap $\psi - \phi$ relationships, and ψ ’s departures from 1, surface different aspects of how structured heads’ contributions interact with the rest.

Track both. They answer different questions, and the gap between them carries the same character of information that η_1 vs. η_2 does at the network level.

6 Summary

Metric	Value-dep.	Loss-dep.	What it measures
G_i, D_i	Yes	Yes (indirectly)	Odometer and displacement per parameter
$\eta_i = D_i /G_i$	Yes	Yes (indirectly)	Per-parameter directedness
$\eta_1 = D _1/G$	Yes	Yes (indirectly)	Network-wide waste (purposefulness)
$\eta_2 = D _2/G$	Yes	Yes (indirectly)	Network-wide geodesic-ness
$\eta_i^{(s)}$	Yes	Yes (indirectly)	Per-parameter per-source directedness
$\sum_s G_i^{(s)}/G_i$	Yes	Yes (indirectly)	Per-parameter source discord
$P_i^{(s)}$	No	Yes	Raw push demand per source (sidecar)
I. Positional Leverage	No	No	Structural potential by graph topology
II. Functional Influence	Yes	No	Realized behavioral influence
III. Structural Potential	No	No	Architectural influence in expectation
$\rho_i(t)$	Yes	No	Realized vs. architectural potential
ϕ	Yes	Yes (indirectly)	Structured share of demanded leveraged motion
ψ	Yes	Yes (indirectly)	Structured demanded vs. committed leveraged motion

Layered structure. The framework composes cleanly across four layers:

1. **Raw motion** (G, D, η) — the trajectory itself.
2. **Attribution** ($G^{(s)}, D^{(s)}, \eta^{(s)}, P^{(s)}$) — who is responsible for the motion.
3. **Leverage** ($\lambda^{\text{pos}}, \lambda^{\text{fn}}, \lambda^{\text{str}}, \rho$) — which motion matters.
4. **Combined** ($B^{(s)}, \tilde{\eta}^{(s)}, \phi, \psi$) — leverage-weighted attribution.

Each layer is independently meaningful. Most of the diagnostic value lives in ratios and gaps rather than absolute numbers — η vs. 1, η_1 vs. η_2 , $\sum_s G^{(s)}$ vs. G , ρ vs. 1, ϕ vs. ψ — and these surface tensions and inefficiencies that scalar loss curves cannot see.